

NutriCoach AI

App fullstack de hábitos, nutrición orientativa e IA

Versión actualizada: PostgreSQL + Sequelize

Resumen ejecutivo

NutriCoach AI permite registrar comidas, consultar un dashboard de progreso, generar menús con IA y estimar de forma orientativa calorías a partir de una imagen del plato. Esta versión sustituye MongoDB/Mongoose por PostgreSQL/Sequelize para trabajar con un modelo relacional más claro, con migraciones, relaciones y datos estructurados.

Tipo de proyecto	Proyecto grupal Full Stack - MVP en 10 días hábiles
Equipo	4 desarrolladores
Repositorio	git@github.com:darenazag/Nutricoach.git
Stack principal	React + TypeScript, Node.js + Express, PostgreSQL + Sequelize
Diferencial	IA para menús, chat y análisis orientativo de imagen de plato

Aviso de alcance seguro

La app debe presentarse como herramienta educativa y de hábitos saludables. No realiza diagnóstico médico, no sustituye a nutricionistas ni profesionales sanitarios y sus estimaciones calóricas son aproximadas.

1. Pitch de la idea

NutriCoach AI es una aplicación fullstack que ayuda a los usuarios a mejorar sus hábitos de alimentación mediante registro de comidas, dashboard de progreso, generación de menús personalizados y un asistente IA para resolver dudas generales sobre nutrición y hábitos saludables.

El elemento diferencial es que el usuario puede subir una imagen de un plato y la IA identifica alimentos probables y porciones aproximadas. Después, la app calcula una estimación orientativa de calorías y macronutrientes, permitiendo al usuario corregir los datos antes de guardarlos.

2. Problema que resuelve

- Muchas personas no tienen visibilidad clara de lo que comen durante el día.
- Calcular calorías manualmente puede ser tedioso.
- Planificar menús semanales requiere tiempo y conocimientos básicos de nutrición.
- Las apps de seguimiento suelen ser complejas para usuarios principiantes.
- El usuario necesita una guía sencilla, visual y orientativa para tomar mejores decisiones.

3. Propuesta de valor

Para usuarios	Para el proyecto	Para la demo
Registro simple de comidas, progreso y menús.	MERN adaptado a SQL con PostgreSQL y Sequelize.	Demo visual: dashboard + IA + imagen de plato.
Asistente IA para dudas y alternativas saludables.	CRUDs claros, relaciones, migraciones y seeders.	Muy fácil de explicar al público.
Estimaciones orientativas, no médicas.	Stack muy vendible para entrevistas.	Flujo wow con foto + recomendación.

4. Cambio técnico principal: PostgreSQL + Sequelize

El proyecto se actualiza para usar PostgreSQL como base de datos relacional y Sequelize como ORM en el backend. Este cambio encaja muy bien con NutriCoach AI porque la app maneja entidades con relaciones claras: usuarios, perfiles, comidas, alimentos, planes de menú e interacciones con IA.

Ventajas para este proyecto

- **Modelo relacional claro:** facilita definir relaciones entre usuarios, comidas, alimentos y planes.
- **Migraciones:** Sequelize permite versionar cambios de base de datos durante el proyecto.
- **Seeders:** podéis cargar datos demo para la presentación.
- **SQL:** es muy valorado por empresas y permite demostrar consultas estructuradas.
- **Consistencia:** PostgreSQL ayuda a mantener integridad mediante claves foráneas y restricciones.

Decisión de arquitectura

Decisión aprobada

Sustituir MongoDB/Mongoose por PostgreSQL/Sequelize. El backend sigue siendo Node.js + Express + TypeScript, pero la persistencia pasa a un modelo relacional.

5. MVP estricto para 10 días

Bloque	Funcionalidad MVP	Criterio de aceptación
Auth	Registro, login y usuario autenticado.	El usuario puede entrar y ver solo sus datos.
Perfil	Datos básicos: edad, peso, altura, objetivo y actividad.	El dashboard calcula un objetivo aproximado.
Comidas	CRUD de comidas diarias.	Se pueden crear, listar, editar y borrar comidas.
Dashboard	Calorías del día, objetivo, macros y evolución.	Los datos cambian cuando se registran comidas.
Menús IA	Generar menú diario o semanal orientativo.	La IA devuelve menú adaptado al objetivo.
Chat IA	Resolver dudas generales de hábitos.	El usuario obtiene respuestas seguras y no médicas.
Foto plato	Subir imagen y estimar alimentos/calorías.	La IA propone alimentos y el usuario puede corregir.

6. Fuera del MVP

- Diagnóstico médico o tratamiento de enfermedades.
- Planes para patologías específicas.
- Recomendaciones para trastornos alimentarios.
- Rutinas deportivas avanzadas tipo entrenador personal.
- Integración con wearables.
- Escáner de código de barras.
- Sistema social o comunidad.
- Pagos o suscripciones.

7. Stack recomendado

Capa	Tecnologías	Notas
Frontend	React + TypeScript + Vite	SPA con rutas, formularios, dashboard y subida de imagen.
UI	CSS Modules o CSS normal + Recharts	Evitar sobrecargar con librerías innecesarias.
Backend	Node.js + Express + TypeScript	API REST modular por dominios.
Base de datos	PostgreSQL	Modelo relacional, claves foráneas e integridad.
ORM	Sequelize + sequelize-cli	Modelos, migraciones y seeders.
IA	Gemini u otro modelo multimodal	Imagen, generación de menús y chat.
Upload	Multer o subida temporal	Guardar imagen localmente o en servicio externo si da tiempo.
Deploy	Docker Compose + VPS + proxy HTTPS	Frontend, backend y PostgreSQL en red controlada.

8. Arquitectura general

La arquitectura mantiene separación clara entre frontend, backend, base de datos y servicio IA. React nunca llama directamente al proveedor de IA: siempre lo hace el backend para proteger credenciales, validar datos y controlar costes.

Flujo recomendado
React -> Express API -> PostgreSQL/Sequelize para datos. Express API -> Servicio IA para análisis de imagen, generación de menús y chat. El frontend solo muestra resultados y permite al usuario corregir las estimaciones.

Componente	Responsabilidad
Frontend React	Interfaz, formularios, dashboard, subida de imagen y chat.
Backend Express	Autenticación, reglas de negocio, validaciones y endpoints REST.
PostgreSQL	Persistencia relacional de usuarios, comidas, alimentos, planes e IA.
Sequelize	Modelos, asociaciones, migraciones y seeders.
Servicio IA	Prompts controlados, análisis de imagen y generación de respuestas.

9. Modelo de datos relacional propuesto

Este modelo está pensado para PostgreSQL + Sequelize. Es más explícito que el modelo documental y facilita demostrar relaciones reales durante la presentación.

Tabla	Campos clave	Relaciones	Uso en la app
users	id, name, email, password_hash, created_at	1:1 health_profiles, 1:N meals, menu_plans, ai_interactions	Cuenta de usuario y autenticación.
health_profiles	id, user_id, age, weight, height, goal, activity_level, daily_calorie_target	N:1 users	Datos base para calcular objetivos.
meals	id, user_id, type, date, title, total_calories, protein, carbs, fat, image_url	N:1 users, 1:N meal_items	Registro de desayuno/comida/cena/snack.
meal_items	id, meal_id, food_name, quantity, unit, calories, protein, carbs, fat	N:1 meals	Detalle de alimentos dentro de cada comida.
menu_plans	id, user_id, title, goal, days_json, generated_by_ai, created_at	N:1 users	Planes diarios o semanales generados por IA.
daily_logs	id, user_id, date, water_liters, weight, notes	N:1 users	Seguimiento diario de peso, agua y notas.
ai_interactions	id, user_id, type, input_json, output_json, created_at	N:1 users	Historial de análisis de imagen, chat y menús.

Relaciones principales

- Un usuario tiene un perfil nutricional.
- Un usuario tiene muchas comidas.
- Una comida tiene muchos alimentos o ingredientes.
- Un usuario puede tener muchos planes de menú.
- Un usuario puede tener muchas interacciones con IA.
- Los logs diarios permiten registrar peso, agua y notas sin mezclarlo con comidas.

10. Endpoints REST propuestos

Módulo	Endpoint	Descripción
Auth	POST /api/auth/register	Crear usuario.
Auth	POST /api/auth/login	Iniciar sesión.
Auth	GET /api/auth/me	Obtener usuario autenticado.
Profile	GET /api/profile/me	Ver perfil nutricional.
Profile	PUT /api/profile/me	Actualizar datos básicos y objetivo.
Meals	GET /api/meals?date=	Listar comidas de un día.
Meals	POST /api/meals	Crear comida manual.
Meals	PUT /api/meals/:id	Editar comida.
Meals	DELETE /api/meals/:id	Eliminar comida.
Dashboard	GET /api/dashboard/me	Resumen diario/semanal.
Menus	POST /api/menus/generate	Generar menú con IA.
Menus	GET /api/menus	Listar menús guardados.
AI	POST /api/ai/analyze-plate	Analizar imagen de plato.
AI	POST /api/ai/chat	Chat de hábitos saludables.

11. Diseño del módulo IA

La IA debe estar encapsulada en un servicio del backend, por ejemplo aiService, con prompts controlados y respuestas estructuradas. El objetivo no es que la IA decida por el usuario, sino que ayude a interpretar, proponer y resumir.

Caso de uso	Input	Output esperado	Control de seguridad
Foto de plato	Imagen + objetivo del usuario	Alimentos probables, porción aproximada, calorías estimadas	Mostrar aviso de estimación y permitir corrección.
Generar menú	Objetivo, calorías, preferencias	Menú diario/semanal estructurado	Evitar dietas extremas y lenguaje médico.
Chat	Pregunta del usuario	Respuesta educativa y prudente	No diagnosticar ni tratar enfermedades.
Resumen diario	Comidas del día	Consejo breve y accionable	Recomendar hábitos generales.

Regla de oro para IA

Toda respuesta relacionada con salud debe ser orientativa, prudente y no médica. Si el usuario pregunta por patologías, medicación, trastornos alimentarios o síntomas, la app debe recomendar consultar a un profesional sanitario.

12. Flujo de demo recomendado

- El usuario inicia sesión.
- Completa perfil: peso, altura, objetivo y actividad.
- El dashboard muestra objetivo diario aproximado.
- Registra una comida manual.
- Sube una foto de un plato.
- La IA detecta alimentos probables y propone una estimación.
- El usuario corrige o confirma la estimación.
- El dashboard se actualiza.
- El usuario pide un menú para mañana.
- El chat IA ofrece una recomendación de hábito segura y práctica.

13. Tabla de trabajo para 4 desarrolladores

Rol	Responsabilidad	Tareas principales	Ramas sugeridas	Entregable visible
Frontend UI/UX	Diseño visual, layout y experiencia.	Landing, dashboard visual, formularios base, responsive, componentes UI.	feat/ui-layout, feat/dashboard-ui	Interfaz navegable y presentable.
Frontend funcional	Conexión de pantallas con API.	Comidas, historial, subida de imagen, menús, gráficas y estados loading/error.	feat/meals-ui, feat/image-upload, feat/menu-ui	Flujos completos desde React.
Backend/API SQL	API REST, Sequelize y PostgreSQL.	Auth, modelos, migraciones, asociaciones, CRUD meals/profile/menu, seeds.	feat/auth-api, feat/sql-models, feat/meals-api	API funcional con DB relacional.
IA + Deploy + Docs	Integración IA, Docker, VPS y documentación.	Servicio Gemini/IA, análisis de imagen, chat, menús, Docker Compose, README y demo script.	feat/ai-service, feat/docker-deploy, docs/readme	IA funcionando y app desplegada.

14. Plan de trabajo en 10 días hábiles

Día	Objetivo	Tareas	Entregable
1	Definición	Cerrar MVP, roles, modelo relacional, issues, wireframes y repo.	README inicial + tablero.
2	Base técnica	React/Vite, Express/TS, PostgreSQL, Sequelize, Docker dev opcional.	Frontend y backend arrancan.
3	Auth + perfil	Registro/login, users, health_profiles, middleware auth.	Usuario autenticado con perfil editable.
4	Comidas API	migrations meals/meal_items, CRUD, validaciones y seeds.	CRUD probado con Postman/Thunder.
5	Comidas UI	Formulario comidas, historial diario y conexión con API.	Registro manual funcional.
6	Dashboard	Resumen diario, calorías, macros, gráficas y checklist.	Dashboard con datos reales.
7	IA menús/chat	Servicio IA, prompts, endpoints menus/chat.	IA genera menú y responde dudas.
8	IA imagen	Upload, analyze-plate, corrección de estimación y guardado.	Demo wow de foto del plato.
9	Deploy + QA	Docker Compose, VPS, variables, HTTPS, CORS, logs.	URL pública funcionando.
10	Presentación	README final, memoria, capturas, demo script y ensayo.	Presentación lista.

15. Flujo Git recomendado

El repositorio indicado es `git@github.com:darenazag/Nutricoach.git`. La rama por defecto actual es `main`. Para trabajar en equipo de forma segura, se recomienda crear `dev` como rama de integración y proteger `main` si es posible.

Estrategia de ramas

`main` = estable/demo. `dev` = integración. `feat/*` = funcionalidades. `fix/*` = correcciones. `docs/*` = documentación. Los Pull Requests se hacen contra `dev` y solo se mergea a `main` cuando la versión esté probada.

Rama	Uso	Ejemplo
<code>main</code>	Versión estable para demo/despliegue.	Merge desde <code>dev</code> al final de cada hito.
<code>dev</code>	Integración diaria del equipo.	PRs revisadas desde <code>feat/*</code> .
<code>feat/*</code>	Nueva funcionalidad.	<code>feat/meals-api</code> , <code>feat/dashboard-ui</code> .
<code>fix/*</code>	Corrección concreta.	<code>fix/cors-deploy</code> .
<code>docs/*</code>	README, memoria y presentación.	<code>docs/readme-initial</code> .

16. Deploy con PostgreSQL

El despliegue en VPS debe contener al menos frontend, backend y base de datos PostgreSQL. PostgreSQL no debe exponerse públicamente; solo debe ser accesible desde el backend dentro de la red Docker.

Servicio	Responsabilidad	Notas de seguridad
<code>nutricoach_frontend</code>	Aplicación React compilada.	Servida por Nginx/contenedor y HTTPS.
<code>nutricoach_backend</code>	API Express.	Variables protegidas y CORS limitado.
<code>nutricoach_postgres</code>	Base de datos PostgreSQL.	Sin puertos públicos; red interna Docker.
<code>proxy</code>	Nginx Proxy Manager o Nginx.	HTTPS y dominio/subdominio.

Variables de entorno orientativas

Variable	Uso
NODE_ENV	production/development
PORT	Puerto del backend
DATABASE_URL	postgres://user:password@host:5432/nutricoach
JWT_SECRET	Firma de tokens/sesiones
CLIENT_URL	URL pública del frontend
GEMINI_API_KEY	Clave IA solo en backend
GEMINI_MODEL	Modelo usado por el servicio IA

17. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Tema salud sensible	Puede parecer diagnóstico médico.	Usar lenguaje de hábitos, aviso legal y respuestas prudentes.
Estimación por imagen imprecisa	La IA puede fallar en porciones.	Permitir corrección manual y mostrar aproximación.
Demasiadas features	No llegar al MVP.	Primero comidas manuales + dashboard; imagen como extra wow.
IA tarda o falla	Demo inestable.	Guardar ejemplos fallback y manejar errores.
Deploy con PostgreSQL	Problemas de conexión/env.	Docker Compose, logs y variables documentadas desde el día 2.

18. Checklist de verificación

Funcional	Técnico	Presentación
Login funciona. Perfil editable. CRUD comidas. Dashboard actualiza datos. Menú IA generado. Chat IA responde. Imagen de plato analizada.	PostgreSQL conecta. Migraciones ejecutan. Seeders cargan datos demo. Rutas protegidas. CORS correcto. API key solo backend. Build frontend/backend correcto.	Demo con usuario preparado. Datos de ejemplo. Capturas. README actualizado. Script de demo. Roles claros. Futuras mejoras definidas.

19. Futuras mejoras

- Escáner de código de barras con Open Food Facts u otra fuente.
- Integración con base nutricional más completa.
- Exportar informes semanales.
- Rutinas de ejercicio básicas según objetivo.
- Preferencias alimentarias avanzadas.
- Panel admin para datos demo y métricas.
- Modo familiar o multiusuario.
- App móvil futura.